

LLM 意圖導向防釣魚瀏覽器擴充功能 (Intent-Based Phishing Detector)

專案開發架構與期末進度指南

壹、系統架構與技術棧 (System Architecture)

本系統旨在防禦「武器化文本」的社交工程攻擊，拋棄傳統的黑名單機制，改以 LLM 即時進行語意與意圖分析 (Intent Analysis)。

1. 前端 (Chrome Extension - Manifest V3)
 - **Content Script**: 靜默讀取當前網頁 (如 Gmail 內文、社群網站私訊) 的純文本。
 - **UI/UX**: 接收到高風險警告時，在網頁 DOM 注入紅色警告遮罩 (Warning Overlay)，並顯示 LLM 的「白話文防詐解釋」。
2. 後端 (Python FastAPI)
 - 作為中繼站，接收前端傳來的文本，避免將 LLM API Key 暴露在前端。
 - 負責請求排隊與快取 (Caching)，避免重複分析相同的網頁。
3. LLM 引擎 (AI Engine)
 - **Baseline (對照組)**: 呼叫 OpenAI (gpt-4o-mini) 或 Anthropic (Claude 3 Haiku) 搭配 Few-Shot Prompting。
 - **Fine-tuned Model (實驗組/拿分亮點)**: 使用 LoRA 技術微調的輕量級開源模型 (如 Llama-3-8B 或是 TAIDE)，部署於 Google Colab 或本地端。

貳、團隊分工與「個人報告」撰寫策略

為了符合專案要求「個人對特定子系統有明確所有權 (distinct ownership)」，並讓 Week 14/15 容易寫出 3-6 頁的報告，強烈建議採取以下 **A、B、C** 鐵三角分工：

組員 A: 前端與系統整合工程師 (Frontend & Integration)

- 職責: 負責 Extension 開發、DOM 擷取演算法、警告畫面渲染、以及 FastAPI 後端架設。
- **Week 14 個人報告重點 (原型與環境)**:
 - Chrome Extension Manifest V3 環境建置過程與權限設計。
 - 討論如何精準擷取 DOM 文本 (例如如何只抓信件內文，過濾掉不相干的 HTML 標籤與雜訊)。
 - 附上 FastAPI 基礎架構程式碼與 API 規格文件。
- **Week 15 個人報告重點 (優化與實作)**:
 - 非同步處理 (Async) 與延遲優化: 如何讓使用者在瀏覽網頁時感覺不到卡頓。
 - UI/UX 設計: 探討如何設計出有效吸引使用者注意、且不會造成「警報疲勞」的注入式警告視窗 (附上截圖與程式碼)。

組員 B: 資料工程與模型微調研究員 (Data & Fine-tuning)

- 職責: 專注於收集台灣在地化的釣魚資料、資料清洗, 以及執行 LoRA 模型微調腳本。(這是拿高分的關鍵技術)
- **Week 14 個人報告重點 (初期研究與資料):**
 - 研究目前常見的社交工程手法 (參考 Week 12 投影片)。
 - 記錄如何使用爬蟲或公開資料庫, 收集初期的 100 筆釣魚信件與正常信件資料。
 - 資料清理 (Data Cleansing) 的過程: 如何將文本轉為 {"text": "...", "label": "phishing", "intent": "urgency"} 的 JSONL 格式。
- **Week 15 個人報告重點 (微調腳本與訓練記錄):**
 - 詳細記錄在 Google Colab 上架設 Unsloth / HuggingFace 進行 LoRA 微調的過程。
 - 放上訓練過程的 Loss 曲線圖 (Training Loss Charts)。
 - 探討遇到的 GPU 記憶體限制問題及如何解決 (如 4-bit 量化技術)。

組員 C: 提示詞工程與資安評估分析師 (Prompt & Security Evaluation)

- 職責: 設計意圖分析的 Prompt 框架、定義威脅模型 (Threat Model)、建立測試資料集並產出量化評估指標。
- **Week 14 個人報告重點 (Prompt 原型與威脅模型):**
 - 定義系統要抓取的四大意圖 (Urgency, Authority, Financial, Greed) 及其資安學理依據。
 - 記錄 Prompt Engineering 的迭代過程 (第一版 Prompt 為什麼會失敗/誤判? 第二版加入了 Few-shot 後改善了什麼? 附上對話截圖)。
- **Week 15 個人報告重點 (評估指標 Evaluation Metrics):**
 - 建立一份絕對未經訓練的「測試資料集 (Test Set)」。
 - 跑出並解釋評估數據: 準確率 (Accuracy)、精確度 (Precision)、召回率 (Recall) 和 F1-Score。
 - 繪製並解釋「混淆矩陣 (Confusion Matrix)」, 進行 False Positive (誤判為釣魚) 與 False Negative (漏抓) 的案例深度檢討。

參、專案里程碑時間軸 (Milestone Timeline)

Week 14: 環境建置與模組原型 (個人 Checkpoint)

- 團隊共同目標: 確定題目、分工, 並產出一個「能在終端機裡動起來」的最簡可行產品 (MVP)。
- 實作進度:
 - A 寫出能抓取網頁文字並發送 API 的擴充功能原型。
 - B 整理出至少 100 筆可用於測試的 CSV/JSONL 資料。
 - C 完成能在 Postman 或 Python 裡穩定吐出 JSON 格式風險評估的 Prompt。
- 繳交: 3-6 頁個人報告 (強調自己的研究軌跡與除錯過程)。

Week 15: 模型評估與深度優化 (個人 Checkpoint)

- 團隊共同目標: 系統整合, 並產出硬核的「資安量化數據」。
- 實作進度:
 - A 將系統串接完成, 當 LLM 判定為高風險時, 擴充功能能成功跳出警告。
 - B 執行模型微調 (Fine-tuning) 腳本, 訓練出專屬的在地化小模型 (如果時間不允許微調, 則 B 可轉為實作 RAG 知識庫, 讓 LLM 比對最新的詐騙新聞)。

- C 大量跑測資, 整理出系統的偵測率與誤判率表格。
- 繳交: 3-6 頁個人報告 (此週重點是數據、圖表、腳本與效能探討)。

Week 16: 最終整合與 PoC 展示 (團隊最終死線)

- 團隊共同目標: 爭取 Outstanding (O) 評級, 觸發成績追溯升級。
- 最終報告結構建議:
 1. 摘要 & 痛點分析: 點出傳統黑名單的弱點與 LLM 意圖分析的優勢。
 2. 系統設計 (**System Design**): 畫出架構圖, 說明 Extension -> FastAPI -> LLM 的資料流。
 3. 模型架構與微調 (**Model Integration**): 說明你們選擇的 Prompting 策略與 Fine-tuning 過程。
 4. 安全評估結果 (**Security Evaluation**): 秀出 Week 15 的混淆矩陣與 F1-Score, 證明這不是玩具, 而是有防禦實效的工具。
 5. 架構限制 (**Architectural Limitations**): 誠實討論缺點 (例如: 如果駭客把釣魚文字做成「圖片」, 目前的擴充功能抓不到; 或 LLM API 延遲問題)。

 拿高分小撇步: 在 Week 16 報告中, 多引用 Week 12 投影片中的關鍵字, 例如:
"We shifted from rule-based signature detection to **dynamic behavioral/intent understanding**." (我們從基於規則的特徵偵測, 轉向動態的行為/意圖理解), 這會讓
評分教授覺得你們非常用心融合了課程核心。